

Github, 달력 출력 프로그램

프로그래밍 입문

6/12

2026764044

정원준

<https://github.com/Furihe/python>

pythonPublic

Pin

Watch0

Fork0

Star0

main

1 Branch

0 Tags

Q

Go to file

T

Add file

<>

Code

Furihe	Update readme.md	a69a570 · 12 minutes ago	🕒 92 Commits
0306	Update readme.md	41 minutes ago	
0313	Update readme.md	35 minutes ago	
0320	Update README.md	31 minutes ago	
0327	Update readme.md	27 minutes ago	
0403	Create readme.md	2 months ago	
0410	Update readme.md	20 minutes ago	
0417	Update readme.md	21 minutes ago	
0508	Update readme.md	15 minutes ago	
0515	Update readme.md	12 minutes ago	
README.md	Initial commit	3 months ago	

README

python

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors1

Furihe

Languages

Python100.0%

Week 1

Name	Last commit message	Last commit date
 ..		
 hello.py	Add files via upload	3 months ago
 readme.md	Update readme.md	45 minutes ago
 예제 문제 수정버전.hwp	Add files via upload	3 months ago
 제1장 파이썬 소개(강의).pdf	Add files via upload	3 months ago
 추가문제.hwp	Add files via upload	3 months ago
 테스트문제.py	Update 테스트문제.py	3 months ago

1장 프로그래밍과 파이썬 소개

파이썬의 특징

- 생산성이 뛰어나다.
- 간결하면서도 효율적인 프로그램을 빠르게 작성가능하다.

컴파일러

- 인간이 프로그래밍 언어를 배워서 프로그램을 작성하면 컴파일러(또는 인터프리터)라고 하는 통역 소프트웨어가 프로그램을 기계어로 바꾸어준다.

Week 1

Commits on Jun 12, 2026				
Update readme.md	Furihe authored 1 hour ago	Verified	d565d57	  
Update readme.md	Furihe authored 1 hour ago	Verified	4449b2b	  
Commits on Mar 14, 2026				
Add files via upload	Furihe authored on Mar 14	Verified	bd7b452	  
Add files via upload	Furihe authored on Mar 14	Verified	61edd6d	  
Commits on Mar 13, 2026				
Add files via upload	Furihe authored on Mar 13	Verified	7143349	  
Update 테스트문제.py	Furihe authored on Mar 13	Verified	b891135	  
Add files via upload	Furihe authored on Mar 13	Verified	5f7f042	  
Commits on Mar 12, 2026				
Add files via upload	Furihe authored on Mar 12	Verified	1164a6f	  
Commits on Mar 6, 2026				
Create readme.md	Furihe authored on Mar 6	Verified	52ec834	  

Week 2

.. readme.md 제2장 변수와 수식(강의).pdf 챕터 2 예제 문제.hwp 챕터 2 추가 문제.hwp	2장 변수와 수식 1. 변수 데이터를 저장하는 공간으로 값을 저장하고 변경할 수 있다.	Update readme.md via upload via upload via upload	41 minutes ago 3 months ago 3 months ago 3 months ago
	2. 자료형 - 정수(int) - 실수(float) - 문자열(str) 3. 입력과 출력 - input() : 사용자 입력 - print() : 결과 출력 4. 형 변환 자료형을 다른 자료형으로 변환한다. - int() - float() - str()	5. 주의해야 할 점 - 변수명은 숫자로 시작할 수 없다. - 예약어는 변수명으로 사용할 수 없다. 📌 핵심 정리 - 변수는 데이터를 저장하는 공간이다. - 자료형에는 정수, 실수, 문자열 등이 있다. - input()으로 입력받고 print()로 출력한다.	

Week 2

Commits

History for python / 0313 on main

All usersAll time

Commits on Jun 12, 2026

Update readme.md

Verified

fb73ce7

1 hour ago

Commits on Mar 20, 2026

Add files via upload

Verified

63b6cf6

on Mar 20

Add files via upload

Verified

cbff41a

on Mar 20

Add files via upload

Verified

ce2aacf

on Mar 20

Commits on Mar 6, 2026

Create readme.md

Verified

dfe5d26

on Mar 6

End of commit history for this file

Week 3

..		
Exercise 챕터 3 조건문.hwp	Add files via upload	3 months ago
README.md	Update README.md	39 minutes ago
fact.py	Add files via upload	3 months ago
forexam.py	Add files via upload	3 months ago
leapYear.py	Add files via upload	3 months ago
mstchEx01.py		3 months ago
test.py		3 months ago
제3장 조건문(강의).pdf		
챕터 3 조건문 예제문제.hwp		
챕터 3 프로그래밍 문제.hwp		

3장 조건문

1. 제어문과 제어 구조

프로그램의 실행 흐름을 제어하는 문장을 제어문이라고 한다.

- 순차 구조 : 위에서 아래로 순서대로 실행
- 선택 구조 : 조건에 따라 다른 코드 실행
- 반복 구조 : 같은 코드를 여러 번 실행

2. 관계 연산자

두 값을 비교하여 참(True) 또는 거짓(False)을 반환하는 연산자이다.

3. 논리 연산자

여러 조건을 함께 사용할 때 사용하는 연산자이다.

- and : 모든 조건이 참일 때 참
- or : 조건 중 하나라도 참이면 참
- not : 결과를 반대로 바꿈

4. 조건문(if)

조건식의 결과에 따라 실행할 코드를 결정한다.

- if : 조건이 참일 때 실행
- if-else : 참과 거짓에 따라 다른 코드 실행
- if-elif-else : 여러 조건 중 하나를 선택하여 실행

5. 중첩 조건문

조건문 안에 또 다른 조건문을 넣어 사용할 수 있다. 복잡한 조건을 처리할 때 사용한다.

6. 조건 연산자

간단한 조건문을 한 줄로 표현할 수 있다.

값1 if 조건식 else 값2

7. 주의해야 할 점

조건식 뒤에는 반드시 콜론(:)을 붙여야 한다. 조건문 내부의 코드는 들여쓰기를 해야 한다. 같은 블록의 들여쓰기 깊이는 동일해야 한다. 관계 연산자 =와 ==를 혼동하지 않도록 주의한다.

📌 핵심 정리

조건문은 조건의 참(True), 거짓(False)에 따라 실행 흐름을 결정한다. 관계 연산자와 논리 연산자를 이용해 조건식을 작성한다. if, if-else, if-elif-else를 사용하여 다양한 조건을 처리할 수 있다. 조건문에서는 콜론(:)과 들여쓰기가 매우 중요하다.

Week 3

Commits on Mar 27, 2026

Add files via upload

Furihe authored on Mar 27

Verifiedbcaaa1e

Add files via upload

Furihe authored on Mar 27

Verifieda630699

Add files via upload

Furihe authored on Mar 27

Verifiedafdb70e

Add files via upload

Furihe authored on Mar 27

Verified99c35cb

Add files via upload

Furihe authored on Mar 27

Verified78d42b3

Add files via upload

Furihe authored on Mar 27

Verified826590b

Add files via upload

Furihe authored on Mar 27

Verifieda30c959

Add files via upload

Furihe authored on Mar 27

Verified78ddb3d

Add files via upload

Furihe authored on Mar 27

Verified19297c5

Commits on Mar 20, 2026

Create README.md

Furihe authored on Mar 20

Verified6a991f6

Commits on Jun 12, 2026

Update README.md

Furihe authored 1 hour ago

Update README.md

Furihe authored 1 hour ago

Week 4

Name	Last commit message	Last commit date
 ..		
 4챕터 반복문 예제문제.hwp	Add files via upload	2 months ago
 readme.md	Update readme.md	37 minutes ago
 제4장 반복문(강의).pdf	Add files via upload	2 months ago
 챕터 4 Programming.hwp	Add files via upload	2 months ago
 챕터4 Exercise.hwp	Add files via upload	2 months ago

4장 반복문

1. 반복문의 개념

반복문은 같은 작업을 여러 번 수행할 때 사용하는 제어문이다. 코드를 반복해서 작성하지 않고 효율적으로 프로그램을 작성할 수 있다.

2. for 반복문

정해진 횟수만큼 반복할 때 사용한다.

range() 함수와 함께 자주 사용한다. 리스트, 문자열 등의 데이터를 순서대로 처리할 수 있다.

3. range() 함수

일정한 범위의 숫자를 생성하는 함수이다.

ex)

- range(5) : 0~4
- range(1, 6) : 1~5
- range(1, 10, 2) : 1, 3, 5, 7, 9

4. while 반복문

조건이 참(True)인 동안 계속 반복한다.

반복 횟수가 정해져 있지 않을 때 사용한다. 조건이 거짓(False)이 되면 반복을 종료한다.

5. 중첩 반복문

반복문 안에 또 다른 반복문을 사용하는 방식이다.

6. 반복문 제어

반복문의 실행을 제어하는 명령어이다.

- break : 반복문을 즉시 종료
- continue : 현재 반복을 건너뛰고 다음 반복 실행

7. 무한 반복문

종료 조건이 없으면 반복문이 계속 실행될 수 있다.

- while True: 형태로 작성한다. 반드시 종료 조건을 함께 고려해야 한다.

8. 주의해야 할 점

- 반복문 뒤에는 반드시 콜론(:)을 붙여야 한다.
- 반복문 내부는 들여쓰기를 해야 한다.
- while 문은 종료 조건을 잘 설정해야 한다. 무한 반복문이 발생하지 않도록 주의한다.

핵심 정리

반복문은 같은 작업을 여러 번 수행할 때 사용한다.

- for 문은 정해진 횟수만큼 반복한다.
- while 문은 조건이 참인 동안 반복한다.
- break는 반복을 종료하고 continue는 다음 반복으로 넘어간다.
- 반복문에서는 콜론(:)과 들여쓰기가 중요하며, 무한 반복문에 주의해야 한다.

Week 4

Commits

History for python / 0327 on main

All users

All time

Commits on Jun 12, 2026

Update readme.md

Furihe authored 53 minutes ago

Verified9860969

Commits on Apr 3, 2026

Add files via upload

Furihe authored on Apr 3

Verifiedec78d25

Add files via upload

Furihe authored on Apr 3

Verified55ae518

Add files via upload

Furihe authored on Apr 3

Verified8cbc063

Add files via upload

Furihe authored on Apr 3

Verifiedc7a845d

Commits on Mar 27, 2026

Create readme.md

Furihe authored on Mar 27

Verified7ab52ce

Week 5

Name	Last commit message	Last commit date
📁 ..		
📄 Chapter 5 Exercise.hwp	Add files via upload	2 months ago
📄 Chapter 5 programming.hwp	Add files via upload	2 months ago
📄 Chapter 5 함수 예제문제.hwp	Add files via upload	2 months ago
📄 readme.md	Update readme.md	33 minutes ago
📄 test.py	Create test.py	2 months ago

5장 함수

1. 함수의 개념

- 함수는 특정 작업을 수행하는 명령문들의 집합이다. 자주 사용하는 코드를 함수로 만들어 두면 재사용이 가능하며 프로그램을 효율적으로 작성할 수 있다.

2. 함수 정의

- 함수는 def 키워드를 사용하여 정의한다.
- 함수 이름을 정하고 기능을 구현한다.
- 필요할 때 여러 번 호출하여 사용할 수 있다.

3. 함수 호출

- 정의된 함수를 실행하는 것을 함수 호출이라고 한다.
- 함수 이름 뒤에 괄호 ()를 사용한다.
- 호출해야 함수 내부 코드가 실행된다.

4. 매개변수와 인수

- 함수에 값을 전달하기 위해 사용한다.
- 매개변수(Parameter) : 함수 정의 시 사용하는 변수
- 인수(Argument) : 함수 호출 시 전달하는 실제 값

5. 반환값

- 함수의 실행 결과를 호출한 곳으로 돌려주는 값이다.
- return 문을 사용한다.
- 반환값은 변수에 저장하거나 다른 연산에 사용할 수 있다.

6. 지역 변수와 전역 변수

- 변수의 사용 범위에 따라 구분된다.
- 지역 변수 : 함수 내부에서만 사용 가능
- 전역 변수 : 프로그램 전체에서 사용 가능

7. 내장 함수

- 파이썬에서 기본적으로 제공하는 함수이다.
- print() input() len() max() min() sum() 등이 있다.

8. 주의해야 할 점

- 함수 정의 시 def 뒤에 함수 이름과 괄호를 작성해야 한다.
- 함수 정의 뒤에는 반드시 콜론(:)을 붙인다.
- 함수 내부 코드는 들여쓰기해야 한다.
- 함수를 정의한 후 반드시 호출해야 실행된다.

핵심 정리

- 함수는 특정 기능을 수행하는 코드의 묶음이다.
- def를 사용하여 함수를 정의한다.
- 매개변수와 인수를 통해 데이터를 전달할 수 있다.
- return을 사용하여 결과를 반환할 수 있다.
- 지역 변수와 전역 변수의 차이를 이해해야 한다.
- 함수를 사용하면 코드의 재사용성과 가독성을 높일 수 있다.

Week 5

Commits

History for `python / 0410` on `main`

All users

All time

Commits on Jun 12, 2026

Update readme.md

Furihe authored 47 minutes ago

Verifiedc251659

Update readme.md

Furihe authored 51 minutes ago

Verifiedaee660a

Commits on Apr 17, 2026

Add files via upload

Furihe authored on Apr 17

Verifiedfac08ac

Add files via upload

Furihe authored on Apr 17

Verifiedd52f857

Add files via upload

Furihe authored on Apr 17

Verifiedfabbd2a

Commits on Apr 10, 2026

Create test.py

Furihe authored on Apr 10

Verified4746855

Create readme.md

Furihe authored on Apr 10

Verified33f240a

End of commit history for this file

Week 6

Name	Last commit message	Last commit date
..		
readme.md	Update readme.md	36 minutes ago
챕터 6 추가 문제.hwp	Add files via upload	2 months ago

6장 파이썬 자료구조 1 (리스트)

1. 리스트의 개념

- 리스트는 여러 개의 데이터를 하나의 변수에 저장할 수 있는 자료구조이다. 같은 자료형뿐만 아니라 서로 다른 자료형의 데이터도 함께 저장할 수 있다.

2. 리스트 생성

- 대괄호 []를 사용하여 리스트를 생성한다.
- 여러 개의 값을 저장할 수 있다.
- 각 요소는 쉼표(,)로 구분한다.

3. 인덱스

- 리스트의 각 요소에는 위치 번호가 있다.
- 첫 번째 요소의 인덱스는 0이다.
- 음수 인덱스를 사용하면 뒤에서부터 접근할 수 있다.

4. 리스트 연산

- 리스트의 요소를 추가, 수정, 삭제할 수 있다.
- append() : 요소 추가
- insert() : 원하는 위치에 요소 삽입
- remove() : 요소 삭제
- pop() : 요소를 꺼내며 삭제

5. 슬라이싱

- 리스트의 일부 요소를 추출하는 기능이다.
- 시작 위치와 끝 위치를 지정하여 사용한다.
- 끝 위치는 포함되지 않는다.

6. 리스트 반복 처리

반복문을 이용하여 리스트의 모든 요소를 처리할 수 있다.

- for 문과 함께 자주 사용된다.
- 리스트의 데이터를 순서대로 접근할 수 있다.

7. 리스트 함축(List Comprehension)

- 반복문과 조건문을 사용하여 새로운 리스트를 간단하게 생성하는 방법이다.
- 코드가 간결해진다.

8. 주의해야 할 점

- 인덱스 범위를 벗어나면 오류가 발생한다.
- 삭제된 요소에는 접근할 수 없다.
- 리스트와 문자열의 인덱스는 0부터 시작한다.

📌 핵심 정리

- 리스트는 여러 개의 데이터를 저장하는 자료구조이다.
- 인덱스를 사용하여 요소에 접근할 수 있다.
- append(), insert(), remove(), pop() 등을 사용하여 데이터를 관리한다.
- 슬라이싱으로 원하는 부분만 추출할 수 있다.
- 반복문과 함께 사용하면 리스트의 모든 요소를 처리할 수 있다.
- 리스트 함축을 이용하면 간결하게 리스트를 생성할 수 있다.

Week 6

Commits

History for python / 0417 on main

All users

All time

Commits on Jun 12, 2026

Update readme.md

Furihe authored 48 minutes ago

Verified

d1f31d4

Commits on Apr 24, 2026

Add files via upload

Furihe authored on Apr 24

Verified

a5501be

Create readme.md

Furihe authored on Apr 24

Verified

9f91e01

End of commit history for this file

Week 7

 324p.md			last month
 325p.md			last month
 332p.1.md			last month
 332p.2.md			last month
 334p.md			last month
 335p.md			last month
 343p.md			last month
 349p.md			last month
 350p.md			last month
 351p.md			last month
 352p.2.md			last month
 352p.3.md			last month
 353p.1.md			last month
 p1.md			last month
 p2.md			last month
 p3.md			last month
 p4.md			last month
 p5.md			last month
 p6.md			last month
 p7.md			last month
 p8.md			last month
 p9.md			last month
 pr10.md			last month
 pr11.md			last month
 pr12.md			last month
 pr13.md			last month
 pr14.md			last month
 pr15.md			last month
 pr16.md	Create pr16.md		last month

1. 튜플(Tuple)

튜플은 여러 개의 데이터를 저장하는 자료구조이다.

소괄호 ()를 사용한다. 리스트와 비슷하지만 값을 수정할 수 없다. 데이터가 변경되지 않아야 할 때 사용한다.

2. 튜플 연산

튜플도 인덱싱과 슬라이싱이 가능하다.

인덱스를 이용해 요소에 접근할 수 있다. 요소의 추가, 삭제, 수정은 불가능하다.

3. 딕셔너리(Dictionary)

딕셔너리는 키(Key)와 값(Value)의 쌍으로 데이터를 저장하는 자료구조이다.

중괄호 {}를 사용한다. 키를 이용해 값을 빠르게 찾을 수 있다. 키는 중복될 수 없다.

4. 딕셔너리 활용

딕셔너리에 데이터를 추가, 수정, 삭제할 수 있다.

딕셔너리[키] = 값 : 추가 및 수정

- del : 삭제
- keys() : 키 조회
- values() : 값 조회
- items() : 키와 값 조회

5. 집합(Set)

집합은 중복되지 않는 데이터를 저장하는 자료구조이다.

중괄호 {}를 사용한다. 중복된 값은 자동으로 제거된다. 순서가 없다.

6. 집합 연산

집합은 수학의 집합 연산을 지원한다.

- 합집합 : |
- 교집합 : &
- 차집합 : -

7. 자료구조 비교

- 리스트(List) : 수정 가능, 순서 있음
- 튜플(Tuple) : 수정 불가, 순서 있음
- 딕셔너리(Dictionary) : 키와 값으로 저장
- 집합(Set) : 중복 불가, 순서 없음

8. 주의해야 할 점

- 튜플은 생성 후 값을 변경할 수 없다.
- 딕셔너리의 키는 중복될 수 없다.
- 집합은 인덱스를 사용할 수 없다.
- 집합은 데이터의 저장 순서를 보장하지 않는다.

📌 핵심 정리

- 튜플은 수정이 불가능한 자료구조이다.
- 딕셔너리는 키와 값의 쌍으로 데이터를 저장한다.
- 집합은 중복을 허용하지 않는다.
- 딕셔너리는 빠른 데이터 검색에 유용하다.
- 집합은 합집합, 교집합, 차집합 연산을 지원한다.
- 리스트, 튜플, 딕셔너리, 집합의 특징을 구분할 수 있어야 한다.

Week 7

Create p10.md	Verified	1e7df16			
Furihe authored last month					
Create p9.md	Verified	e26fa2c			
Furihe authored last month					
Create p8.md	Verified	9fc44b1			
Furihe authored last month					
Create p7.md	Verified	fdde661			
Furihe authored last month					
Create p6.md	Verified	7f7f994			
Furihe authored last month					
Create p5.md	Verified	a2e1bf3			
Furihe authored last month					
Rename 3.md to p3.md	Verified	a80c611			
Furihe authored last month					
Rename 2.md to p2.md	Verified	6f1f634			
Furihe authored last month					
Rename 1.md to p1.md	Verified	8124efd			
Furihe authored last month					
Rename 4.md to p4.md	Verified	f809dbe			
Furihe authored last month					
Create 4.md	Verified	41e3de3			
Furihe authored last month					
Create 3.md	Verified	e1fb72a			
Furihe authored last month					
Create 2.md	Verified	12d2b85			
Furihe authored last month					
Create 1.md	Verified	430f46b			
Furihe authored last month					
Furihe authored last month					
Create 352p.md	Verified	5555b59			
Furihe authored last month					
Create 351p.md	Verified	b05aab0			
Furihe authored last month					
Create 350p.md	Verified	9a74926			
Furihe authored last month					
Create 349p.md	Verified	2484800			
Furihe authored last month					
Create 343p.md	Verified	cc350f8			
Furihe authored last month					

Commits on May 14, 2026

Create 335p.md	Verified	e86c465			
Furihe authored last month					
Create 334p.md	Verified	1100f41			
Furihe authored last month					
Create 332p.2.md	Verified	80898ef			
Furihe authored last month					
Create 332p.1.md	Verified	b40a803			
Furihe authored last month					
Create 325p.md	Verified	a1ca8c1			
Furihe authored last month					
Create 324p.md	Verified	3818eb5			
Furihe authored last month					

Commits on Jun 12, 2026

Update readme.md	Verified	37cb370			
Furihe authored 45 minutes ago					

Commits on May 15, 2026

Create pr16.md	Verified	3adb352			
Furihe authored last month					
Create pr15.md	Verified	581c0b6			
Furihe authored last month					
Create pr14.md	Verified	134fed0			
Furihe authored last month					
Create pr13.md	Verified	b762c30			
Furihe authored last month					
Create pr12.md	Verified	73275ee			
Furihe authored last month					
Create pr11.md	Verified	1ad095c			
Furihe authored last month					
Rename p10.md to pr10.md	Verified	d690444			
Furihe authored last month					
Create p10.md	Verified	1e7df16			
Furihe authored last month					

Week 8

Name	Last commit message	Last commit date
..		
p01.md	Create p01.md	3 weeks ago
p02.md	Create p02.md	3 weeks ago
p03.md		3 weeks ago
p04.md		3 weeks ago
p05.md		3 weeks ago
p06.md		3 weeks ago
p07.md		3 weeks ago
p08.md		3 weeks ago
readme.md		1 minutes ago

8장 객체와 클래스

1. 객체지향 프로그래밍

객체지향 프로그래밍(Object-Oriented Programming, OOP)은 데이터와 기능을 하나로 묶어 프로그램을 구성하는 방법이다. 프로그램을 객체 단위로 설계한다. 코드의 재사용성과 유지보수성을 높일 수 있다.

2. 객체(Object)

객체는 데이터와 기능을 함께 가지고 있는 실체이다.

- 속성(Attribute) : 객체가 가지고 있는 데이터
- 메서드(Method) : 객체가 수행하는 기능

예) 자동차 객체

속성 : 색상, 속도 메서드 : 출발하기, 정지하기

3. 클래스(Class)

클래스는 객체를 만들기 위한 설계도이다.

객체의 구조와 기능을 정의한다. 하나의 클래스로 여러 객체를 생성할 수 있다.

📌 핵심 정리

- 객체는 데이터와 기능을 함께 가지는 실체이다.
- 클래스는 객체를 만들기 위한 설계도이다.
- 객체를 생성하는 과정을 인스턴스화라고 한다.
- 생성자 `init()`는 객체 생성 시 자동으로 호출된다.
- `self`는 현재 객체 자신을 의미한다.
- 객체지향 프로그래밍은 코드의 재사용성과 유지보수성을 높여준다.

4. 객체 생성

클래스를 이용해 객체를 생성하는 과정을 인스턴스화(Instantiation)라고 한다.

생성된 객체를 인스턴스(Instance)라고 한다. 하나의 클래스로 여러 개의 객체를 만들 수 있다.

5. 생성자

생성자는 객체가 생성될 때 자동으로 호출되는 특별한 메서드이다.

`init()` 메서드를 사용한다. 객체의 초기값을 설정한다.

6. 멤버 변수와 메서드

클래스 내부에는 데이터와 기능을 정의할 수 있다.

- 멤버 변수 : 객체의 데이터를 저장
- 메서드 : 객체의 기능을 정의

7. self

`self`는 현재 객체 자신을 가리키는 참조 변수이다.

클래스 내부에서 객체의 속성에 접근할 때 사용한다. 메서드의 첫 번째 매개변수로 작성한다. 8. 주의해야 할 점

- 클래스 이름은 일반적으로 첫 글자를 대문자로 작성한다.
- 객체를 생성해야 클래스의 기능을 사용할 수 있다.
- 메서드의 첫 번째 매개변수는 보통 `self`를 사용한다.
- 생성자는 `init()` 형태로 작성한다.

Week 8

Commits on Jun 12, 2026	
Update readme.md	Verified a69a570
Furihe authored 42 minutes ago	
Commits on May 21, 2026	
Update p08.md	Verified 05aa515
Furihe authored 3 weeks ago	
Create p08.md	Verified 59daef5
Furihe authored 3 weeks ago	
Create p07.md	Verified 40d09e7
Furihe authored 3 weeks ago	
Create p06.md	Verified 5d6f057
Furihe authored 3 weeks ago	
Update p05.md	Verified 3decd33
Furihe authored 3 weeks ago	
Create p05.md	Verified ee5d222
Furihe authored 3 weeks ago	
Create p04.md	Verified b9507de
Furihe authored 3 weeks ago	
Create p03.md	Verified 85a36e0
Furihe authored 3 weeks ago	
Create p02.md	Verified dbaac32
Furihe authored 3 weeks ago	
Create p01.md	Verified e8edc16
Furihe authored 3 weeks ago	
Create readme.md	Verified cd99958
Furihe authored 3 weeks ago	

달력 출력 프로그램

연도 2026 월 4 달력 출력

연도 2024 월 4 달력 출력

월	화	수	목	금	토	일
		1 음 3/23	2 음 3/24	3 음 3/25	4 음 3/26	5 음 3/27
6 음 3/28	7 음 3/29	8 음 4/1	9 음 4/2	10 음 4/3	11 음 4/4	12 음 4/5
13 음 4/6	14 음 4/7	15 음 4/8	16 음 4/9	17 음 4/10	18 음 4/11	19 음 4/12
20 음 4/13	21 음 4/14	22 음 4/15	23 음 4/16	24 음 4/17	25 음 4/18	26 음 4/19
27 음 4/20	28 음 4/21	29 음 4/22	30 음 4/23	31 음 4/24		

월	화	수	목	금	토	일
1 음 4/16	2 음 4/17	3 음 4/18	4 음 4/19	5 음 4/20	6 음 4/21	7 음 4/22
8 음 4/23	9 음 4/24	10 음 4/25	11 음 4/26	12 음 4/27	13 음 4/28	14 음 4/29
15 음 5/1	16 음 5/2	17 음 5/3	18 음 5/4	19 음 5/5	20 음 5/6	21 음 5/7
22 음 5/8	23 음 5/9	24 음 5/10	25 음 5/11	26 음 5/12	27 음 5/13	28 음 5/14
29 음 5/15	30 음 5/16					

```

1 import tkinter as tk
2 import calendar
3 from datetime import date
4 from korean_lunar_calendar import KoreanLunarCalendar
5
6 # 양력 날짜를 음력 날짜로 변환하는 함수
7 def get_lunar(year, month, day):
8     lunar = KoreanLunarCalendar()
9
10    # 양력 날짜 설정
11    lunar.setSolarDate(year, month, day)
12
13    # 음력 월, 일 반환
14    return lunar.lunarMonth, lunar.lunarDay
15
16
17 # 달력을 출력하는 함수
18 def show_calendar():
19
20    # 기존에 출력된 달력 삭제
21    for widget in calendar_frame.winfo_children():
22        widget.destroy()
23
24    # 사용자가 입력한 연도와 월 가져오기
25    year = int(year_entry.get())
26    month = int(month_entry.get())
27
28    # 요일 목록
29    days = ["월", "화", "수", "목", "금", "토", "일"]
30
31

```

```

32 # 요일 출력
33 for col, d in enumerate(days):
34     tk.Label(
35         calendar_frame,
36         text=d,
37         width=12,
38         height=2,
39         relief="solid",
40         bg="lightgray"
41     ).grid(row=0, column=col)
42
43 # 해당 월의 달력 데이터 생성
44 month_data = calendar.monthcalendar(year, month)
45
46 # 오늘 날짜 저장
47 today = date.today()
48
49 # 주 단위로 달력 출력
50 for row, week in enumerate(month_data, start=1):
51     for col, day in enumerate(week):
52
53         # 빈 칸 처리
54         if day == 0:
55             text = ""
56             bg = "white"

```

```

56
57     else:
58         # 양력을 음력으로 변환
59         lunar_month, lunar_day = get_lunar(
60             year,
61             month,
62             day
63         )
64
65         # 양력 날짜와 음력 날짜 표시
66         text = f"{day}\n음 {lunar_month}/{lunar_day}"
67
68         bg = "white"
69
70         # 오늘 날짜면 노란색으로 강조
71         if (
72             year == today.year
73             and month == today.month
74             and day == today.day
75         ):
76             bg = "yellow"
77
78         # 기본 글자색
79         fg = "black"
80
81         # 토요일은 파란색
82         if col == 5:
83             fg = "blue"

```

```

84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101

```

```

# 일요일은 빨간색
elif col == 6:
    fg = "red"

# 날짜 칸 생성
tk.Label(
    calendar_frame,
    text=text,
    width=12,
    height=5,
    relief="solid",
    justify="center",
    fg=fg,
    bg=bg,
    font=("맑은 고딕", 10)
).grid(row=row, column=col)

```



```

103 # =====
104 # 메인 GUI 생성
105 # =====
106
107 window = tk.Tk()
108
109 # 창 제목 설정
110 window.title("양력 · 음력 달력")
111
112 # 창 크기 설정
113 window.geometry("900x550")
114
115 # 입력 영역 프레임 생성
116 input_frame = tk.Frame(window)
117 input_frame.pack(pady=10)
118
119 # 연도 입력 라벨
120 tk.Label(
121     input_frame,
122     text="연도"
123 ).pack(side="left")
124
125 # 연도 입력창
126 year_entry = tk.Entry(
127     input_frame,
128     width=8
129 )
130 year_entry.pack(side="left")
131
132 # 현재 연도를 기본값으로 설정
133 year_entry.insert(
134     0,
135     str(date.today().year)
136 )
137
138 # 월 입력 라벨
139 tk.Label(
140     input_frame,
141     text="월"
142 ).pack(side="left")
143
144 # 월 입력창
145 month_entry = tk.Entry(
146     input_frame,
147     width=5
148 )
149 month_entry.pack(side="left")
150
151 # 현재 월을 기본값으로 설정
152 month_entry.insert(
153     0,
154     str(date.today().month)
155 )

```

```
157 # 달력 출력 버튼
158 tk.Button(
159     input_frame,
160     text="달력 출력",
161     command=show_calendar
162 ).pack(side="left", padx=10)
163
164 # 달력이 표시될 프레임
165 calendar_frame = tk.Frame(window)
166 calendar_frame.pack()
167
168 # 프로그램 시작 시 현재 달력 자동 출력
169 show_calendar()
170
171 # GUI 실행
172 window.mainloop()
```


느낀점

프로그래밍을 접하며 파이썬이라는 기본적인 것부터 코드의 흐름, 구조와 같은 것들을 배울 수 있어서 유익했다. 조금 뒤쳐지고 이해 못하는 부분도 있었지만 github와 이를 활용하여 정리를 해보고 달력 출력과 같은 프로젝트를 실현해보며 이해할 수 있다.

깃허브를 활용하면서 편리하게 내가 공부했던 것을 열면서 다시 이해할 수 있었다.

또한 이번 프로젝트를 진행하면서 단순히 코드창에서 결과를 보여주는 것과 gui를 활용해 결과를 보여주는 것은 시각적으로 큰 차이가 있는 것 같다. 또한 수업진행과정에서 새로운 프로그램과 그의 활용성을 알고 활용해볼 수 있었다. 이 수업을 계기로 조금이나마 더 나아갈 수 있었던 것 같고 배운 것을 토대로 다른 프로젝트를 도전 해보고 싶다.

점수

Github: 12/15

발표: 15/15

총합: 27/30

감사합니다